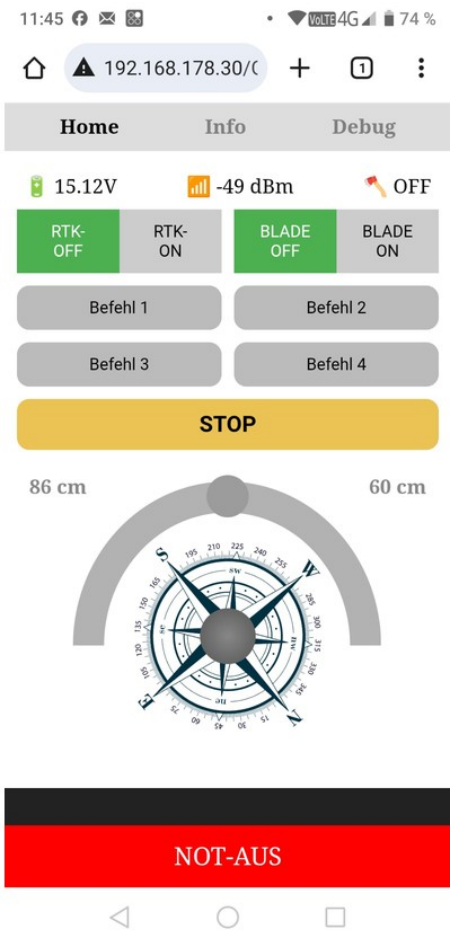
Das ist die von mir und ChatGPT erstellte Webseite. Ich habe sie auf meinem Heimserver abgelegt und rufe sie dann entweder vom Notebook oder was meistens der Fall ist vom Handy auf. Wenn alles auf dem PiMowBot richtig installiert ist, lässt sich der PiMowBot über die Kompassrose in der Mitte Steuern. Mit dem Punkt auf dem Halbkreis um die Kompassrose herum kann man den PiMowbot dann auf der Stelle drehen lassen.



1. Flaskserver Installation

Für den Betrieb der Seite sind auf dem PiMowbot zwei zusätzliche Bibliotheken erforderlich: Flask sowie flask_cors Die Installation habe ich bei mir durchgeführt nachdem ich ein **sudo apt-get update** und **sudo apt-get upgrade** durchgeführt habe. Auch ist eine aktuelle pip Version hilfreich. Da ich auf meinem MINI-PiMowBot noch eine alte pip Version hatte gab es da noch Probleme aber mit
python3 -m pip install pi --upgrade pip konnte ich das dann auch lösen. Danach habe ich
pip install Flask sowie
pip install flask_cors installiert.

Das Script Flaskserver.py habe ich bei meinem MINI- PiMowBot außerhalb des pimowbot Verzeichnisses abgelegt. Dazu habe ich das Verzeichnis /*home/*pi/**Mower/Flaskserver** angelegt und in das Verzeichnis Flaskserver das Flaskserver.py Script hinein kopiert.

```
pi@raspberrypi:~/Mower/Flaskserver $ python Flaskserver.py

Flaskserver erreichbar unter:
http://192.168.178.60:5005

 * Serving Flask app 'Flaskserver'
 * Debug mode: off
```

Wenn python Flaskserver.py dann aus diesem Verzeichnis aufgerufen wird, sollte im Ausgabefenster folgendes sichtbar werden. Ist das der Fall, dann war schon mal alles auf dem Pi erfolgreich.
Damit später auch beim Start vom PiMoBot der Flaskserver gestartet wird, kann man einen Service einrichten. Das beschreibe ich später. Jetzt kümmern wir uns um die html Seite, dazu ist die ausgegebene IP Adresse noch wichtig.

2. HTML Seite installieren

Auf meinem Linux Rechner den ich meistens Nutze habe ich ein Apache2 Webserver laufen. Auf die Installation von Apache falls erforderlich gehe ich nicht weiter ein, dazu gibt es viele Anleitungen im Netz. Ich persönlich würde **nicht** Apache auf dem PiZero im PiMowBot installieren, der PiZero hat mit dem Rebol Webserver und den anderen PiMowBot Scripten schon genügend Auslastung. Ich habe aber auf einem PiZero 2W Nginx als Webserver installiert, das klappte mit meinen diversen Seiten dann auch sehr gut und war sehr flüssig. Auch hierzu gibt es viele Anleitungen. Wichtig, dass die Zugriffsrechte richtig eingestellt sind, damit man an den Dateien auch noch was ändern kann. Gehen wir mal davon aus, dass alles richtig installiert wurde und es gibt jetzt ein Verzeichnis **/var/www/html/remote/** welches der Webserver erreicht. In dieses Verzeichnis muss nun die Datei Mobile.html und rose.png hinein kopiert werden. Im Quellcode der Mobile.html nach dem Eintrag `const PI_IP = "192.168.178.60"; // ggf anpassen!`
suchen und den so anpassen, dass er die IP Adresse des PiMowBots hat. Das wäre alles.
Nun solltest Du die Webseite durch eingabe IP Adresse Deines Server und /*remote*/Mobile.html aufrufen können. Dann sollte die Seite sichtbar sein.

An dieser Stelle hast Du Dir einen Kaffee verdient.

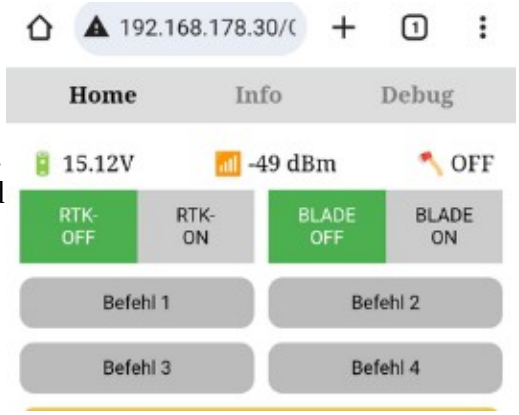
3. Funktion

Damit auf der Webseite die aktuellen Werte der Sensoren angezeigt werden liest der Flaskserver Dateien aus dem /*dev/*shm/ regelmäßig aus.
Dazu gehört der Batterie Ladestand, die Distanzwerte der ToF Sensoren, sowie der Kompass Wert.
Je nach Installation der verwendeten Bauteile können die Quelldateien unterschiedlich heißen und müssen u.U. im Flaskserver.py Script angepasst werden.
Ich habe es möglichst leserlich im Flaskserver.py codiert. Die Dateien werden in der Funktion

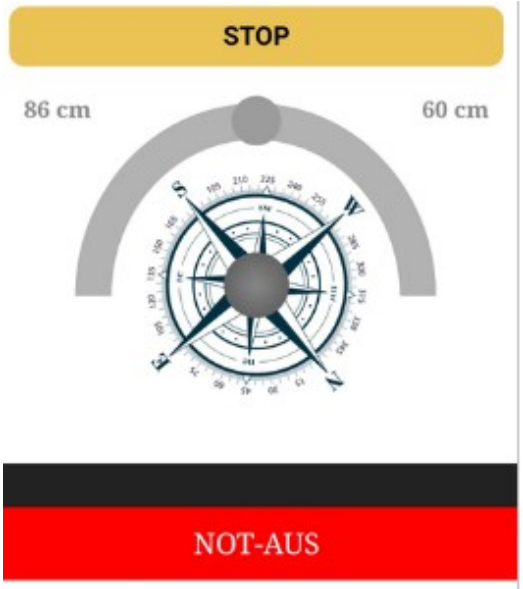
```
def read_data():
        # Daten die übermittelt werden
        #{"Battery": 14.6, "WLAN": "-49 dBm","Blade": False,
        #"kurs": "231", "ToF_l": "84", "ToF_r": "85"}
```

der Reihe nach alle 500ms gelesen und der Variablen data. zugeordnet. Der Flaskserver liefert diese Daten dann nach Aufforderung der html Seite aus.
Ein wenig komplexer ist die Steuerung über den Joystick und dem Wende-Halbkreis. Dieser Part wurde fast ausschließlich von ChatGPT erstellt. Ich habe nur sichergestellt, dass die empfangenen Werte dann für die /dev/shm/.PiMowBot_Motor.left und /dev/shm/.PiMowBot_Motor.right aufbereitet und dort zur Ansteuerung der Motore abgespeichert werden.

4. Bedienung



Die html Seite besteht aus 3 Tabs Home Info und Debug.
Nur der Home Tab ist derzeit mit Elementen versehen. Für meinem Mähroboter habe ich den Plan im Info Tab noch die Karte des Rasens und die Position des Mähers darauf abzubilden.
In der oberen Zeile des Home Tabs befindet sich der Ladezustand des Akkus, der RSSI Wert sowie rechts die Anzeige ob der Messer Motor OFF oder ON ist. Das kann mit dem Switch darunter ein- bzw. ausgeschaltet werden. Den RSSI Wert zu übertragen macht m.E. Sinn, weil es bei mir schon mal vorkam, dass es nicht zu erklärende Fehlfunktionen gab, aber immer dann wenn der RSSI Wert unter -75 lag. Dann war die schlechte Netzverbindung in der Regel der Übeltäter.
Bleibt noch Der Switch RTK OFF und RTK ON. Auch der löst derzeit keine Aktion im Flaskserver aus, bei meinem Mähroboter Starte / Stoppe ich mit diesem Switch den RTK Empfänger.
Auch die Buttons Befehl 1 bis Befehl 4 sind ohne Funktion.



Im unteren Bereiches des Screens wird die Ausrichtung durch die KompassRose angezeigt. Links und rechts sind die Entfernungswerte der ToF Sensoren eingeblendet. Der PiMowBot lässt sich durch betätigen des Runden Kreises in der Mitte in alle Richtungen steuern. Mit dem Punkt auf dem Halbreis, lässt sich der PiMowBot auf der Stelle Links oder Rechts herum drehen.
Der Stop Button lässt alle Antriebs Motore stoppen,
der NOT-Aus Button stoppt auch den Messer Motor, falls er sich dreht. Hält man lange den NOT-Aus Button gedrückt, verfärbt er

sich orange und es wird Shutdown angezeigt. Durch erneutes Betätigen des Shutdowns, wird der Pi im PiMowbot herunter gefahren. Erhält der Flaskserver 20 Sekunden lang keinen Update von dem virtuellen Joystick, dann werden ebenfalls die Antriebsmotore gestoppt.

5. Service für den Flaskserver einrichten

Ist es gewünscht dass der Flaskserver automatisch nach dem starten des Pi's sich aktiviert, dann lässt sich das über die Einrichtung eines Services erzielen.
Hier ein Vorschlag für den Service
mit Eingabe von sudo nano **/etc/systemd/system/Flaskserver.service** erzeugt nano eine neue Servicedatei. Kopiere diesen Inhalt in den Editor

Description=MowerFlaskserver
After=network-online.target
Wants=network-online.target

[Service]
WorkingDirectory=/home/pi/Mower/Flaskserver
ExecStart= /usr/bin/python /home/pi/Mower/Flaskserver/Flaskserver.py
User=pi
Restart=always
RestartSec=2
StandardOutput=append:/home/pi/Mower/Flaskserver/Flaskserver.log
StandardError=append:/home/pi/Mower/Flaskserver/Flaskserver.log
SyslogIdentifier=Flaskserver

[Install]
WantedBy=multi-user.target

Danach mit strg+s speichern und mit strg+x beenden.
Ich persönlich teste dann immer erst den Service. Das Script Flaskserver.py darf jetzt nicht laufen.
Nun mir dem Aufruf s**udo systemctl start Flaskserver.service** dem Service starten.
Ist die html Seite geöffnet erscheint nach wenigen Sekunden „Verbunden" über dem Not-Aus Button. Also dann ist alles richtig eingerichtet.
Zur Kontrolle noch ein **sudo systemctl status Flaskserver.service** dieses sollte folgendes ausgeben

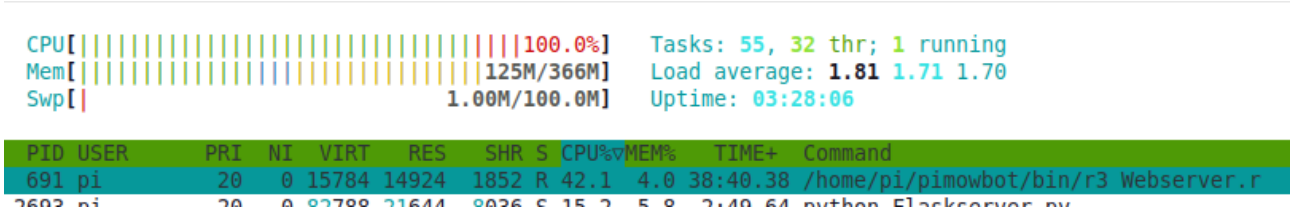
```
pi@raspberrypi:~ $ sudo systemctl status Flaskserver.service
● Flaskserver.service
     Loaded: loaded (/etc/systemd/system/Flaskserver.service; disabled; vendor preset: enabled)
     Active: active (running) since Fri 2026-10-09 14:52:24 CEST; 27s ago
   Main PID: 19212 (python)
      Tasks: 4 (limit: 724)
        CPU: 8.664s
     CGroup: /system.slice/Flaskserver.service
             └─19212 /usr/bin/python /home/pi/Mower/Flaskserver/Flaskserver.py

Okt 09 14:52:24 raspberrypi systemd[1]: Started Flaskserver.service.
Okt 09 14:52:52 raspberrypi systemd[1]: /etc/systemd/system/Flaskserver.service:1: Assignment outsi
```

Das grüne Active (running) zeigt, dass alles richtig eingerichtet wurde.
Mit einem sudo systemctl enable Flaskserver.service wird veranlasst, dass dieser Service nach jedem booten wieder aufgerufen wird.

6. Anmerkungen

Was mir auf meinem Testsystem aufgefallen ist:
In meinem Mini-PiMowbot habe ich eine PiZero W der ersten Generation verbaut und das OS bullseye installiert. Beim Aufruf des Befehls htop sehe ich die Auslastung des CPU Kernes



Nach längerer Laufzeit benötigt der Rebol Webserver eine CPU Last von fast 50%. Dann kommt es schon mal vor, dass die html Fernbedienung nicht zeitnah arbeitet. Den CPU Hunger des Webservers habe ich schon an die Entwicklung weitergegeben. Einen Fix dazu gibt es derzeit noch nicht. Was ich aus Interesse einmal gemacht habe:
Nachdem der Webserver gestartet ist und der PiMowBot kurz die Räder bewegt hat, liegen ja im /dev/shm/Verzeichnis alle erforderlichen Dateien. Diese sind dann auch nach den Einstellungen in der options Datei passend erzeugt worden. Ich habe mir das komplette /dev/shm/Verzeichnis in ein tmp Verzeichnis kopiert und danach den Systemservice zum Starten des PimowBot Webservers disabled. Danach habe ich das System rebootet. Nun werden keine Dateien in das /dev /shm / Verzeichnis hinein kopiert da der Rebol Webserver nicht gestartet ist. Nun habe ich per Hand alles was in meinem tmp Verzeichnis vorhanden war in das /dev/shm Verzeichnis kopiert und danach jedes python script per Hand gestartet. Die manuelle Steuerung des PiMowBots war nun viel flüssiger und das System war nicht mehr so unter last. Damit ist aber natürlich kein Automow möglich. Ein Pi3 mit 4 CPU Kernen ist m.E. viel Leistungsstärker und kommt mit dem Rebol Webserver des PiMowBots besser zurecht. Das habe ich durch Tests verifizieren können.